CHAPTER 3 Approximations and Round-off Errors

- we want to limit errors in our numerical solutions, since they involve approximations.

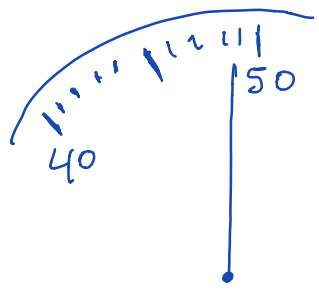
Numerical errors:

Round-off error: computers can only represent numbers with a finite number of digits.

Truncation error: introduced by the numerical methods which use approximations to represent exact mathematical operations.

Significant Figures

EX Speedometer



$\Rightarrow$ 49 km/hr approx.

- we want to know the speed to one decimal place. one may say 48.8, another 48.9
- Limitations of the instrumet $\Rightarrow$ only the first two digits may be used with confidence

Significant digits: of a number are those that can be used with confidence. They correspond to the # of certain digits plus one estimated digit.

- Speedometer : two certain digits 48. It is conventional to set the estimated digit at one half of the smallest scale division on the measurement device

  Three significant figures : 48.5

- Zeros are not always significant figures.

  0.00001845
  0.0001845
  0.001845

  } all have 4 significant figures

  ↳ zeros are locating the decimal pt.

- In large number, it may not be clear how many zeros are significant.

  Ex 45,300 may have 3, 4, or 5 significant digits, if zeros are known with confidence.

- Resolve uncertainty by using scientific notation

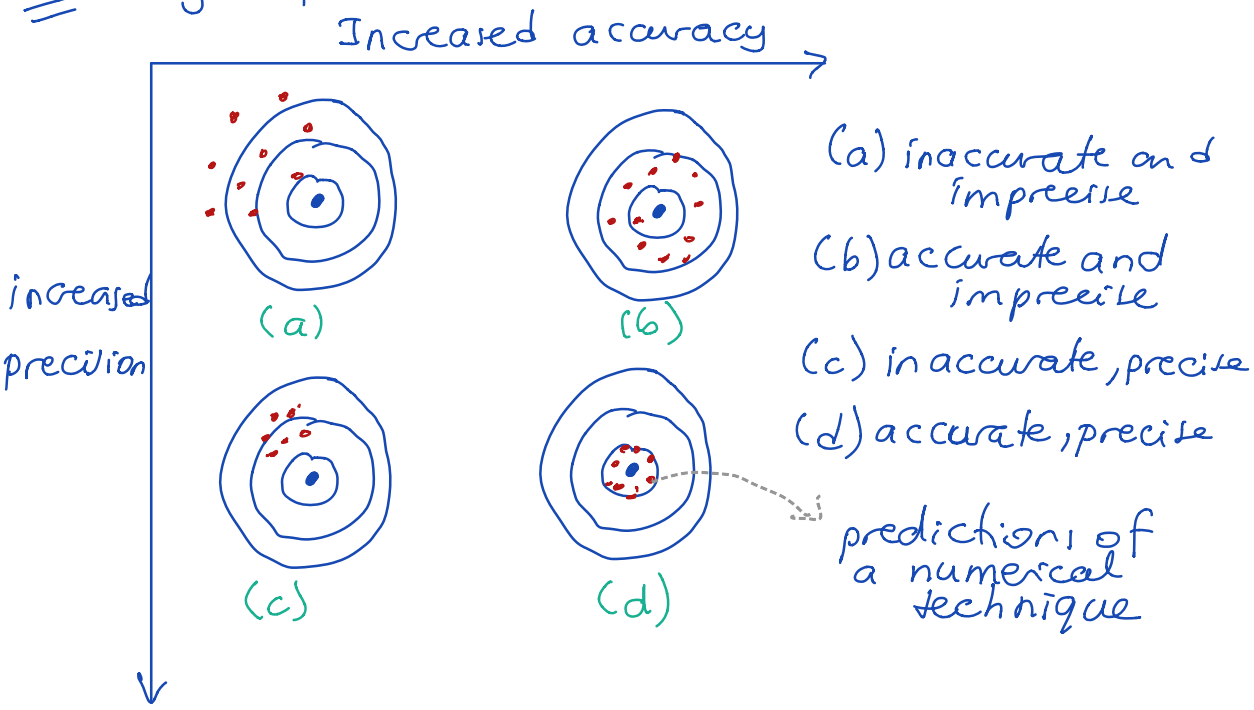
  4.53×10^4 → 3 significant figures
  4.530×10^4 → 4 " "
  4.5300×10^4 → 5 " "

⇒ Importance : You may a numerical solution if it is correct & e.g. 4 significant figures.

Accuracy: How closely a computed/measured value agrees with the true value.

Precision: How closely individual computed/measure values agree with each other.

Ex Target practice



- We want accurate and precise numerical solutions.

error: represents both the inaccuracy and imprecision.

Error Definitions

Truncation errors: result when approximations are used to represent exact mathematical procedures.

Round-off errors: when numbers having limited significant figures are used to represent exact numbers.

True value = approximation + error

E_t = true value − approximation

E_t: true error

We can normalize the error using the true value to account for the magnitudes of quantities:

True fractional relative error = $\frac{\text{true error}}{\text{true value}}$

$$\varepsilon_t = \frac{\text{true error}}{\text{true value}} \cdot 100\,\%$$

↳ true percent relative error

Ex Tasks of measuring the lengths of a bridge and a rivet.

	True values	Measured values
bridge:	10,000cm	9999cm
rivet:	10cm	9cm

Compute a) True error b) true percent relative error

a) $E_t = 10000 - 9999 = 1\text{cm}$ (bridge)

$E_t = 10 - 9 = 1\text{cm}$ (rivet)

b) $\varepsilon_t = \frac{1}{10000}\,100\% = 0.01\,\%$ (bridge)

$\varepsilon_t = \frac{1}{10}\,100\% = 10\,\%$ (rivet)

Conclusion: an adequate job of measuring the bridge, but not the rivet

In real life the true value is unknown.
Alternative method: normalize the error using the best available estimate of the true value:

$$\varepsilon_a = \frac{\text{approximate error}}{\text{approximation}} 100\%$$

approximate value

- In iterative approaches: successively compute better approximations.

Percent relative error:

$$\varepsilon_a = \frac{\text{current approximation} - \text{previous approximation}}{\text{current approximation}} 100\%$$

Repeat iterations until:

$$|\varepsilon_a| < \varepsilon_s$$ → prespecified percent tolerance

It can be shown that if $\varepsilon_s = (0.5 \times 10^{2-n})\%$ (1) then the result is correct to at least n significant digits.

Ex Error estimates for iterative methods

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} + \cdots$$

(Maclaurin Series expansion)

Estimate $e^{0.5}$ by adding terms until ε_a is less than ε_s, conforming to 3 significant digits.

Soln: Use (1) $\varepsilon_s = (0.5 \times 10^{2-3})\% = 0.05\%$

True value: $e^{0.5} = 1.648721\ldots$

First estimate $e^{0.5} \simeq 1$

Second estimate $e^{0.5} \simeq 1 + 0.5 = 1.5$

$$\varepsilon_t = \frac{1.648721 - 1.5}{1.648721} 100\% = 9.02\%$$

$$\varepsilon_a = \frac{1.5 - 1}{1.5} 100\% = 33.3\% \quad \text{not less than } \varepsilon_s$$

Add terms

After 6 iterations the result is accurate to 5 digits

use 6 terms: $e^{0.5} \simeq 1.648697$

$$\varepsilon_t = 0.00142\%$$

$$\varepsilon_a = 0.0158\% < \varepsilon_s = 0.05\%$$

Round-off Errors

- Numbers such as $\pi, e, \sqrt{7}$ can't be represented exactly by a computer.

Computer Representation of Numbers:

binary digits = bits

words: consist of bits

Numbers are typically stored in one or more words.

Number Systems:

Decimal, base-10 system: 10 digits

$8\ 6\ 4\ 0\ 9 = (8\times10^4)+(6\times10^3)+(4\times10^2)+(0\times10^1)+(9\times10^0)$

$10^4\ 10^3\ 10^2\ 10^1\ 10^0$

Binary or base-2 systems:

$11 = (1\times2^1)+(1\times2^0) = 2+1 = 3$

$(1\ 0\ 1\ 0\ 1\ 1\ 0\ 1)_2$

$2^7\ 2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$

$= (1\times128)+(0\times64)+\cdots+(1\times2^0) = (173)_{10}$

Integer Representation

Signed Magnitude Method

→ employ the first bit for the sign: 0 for positive, 1 for negative

→ Remaining bits are used for storing the number

Ex 16-bit computer; representation of −173

1 0 0 0 0 0 0 0 1 0 1 0 1 1 0 1

sign — number (15 bits)

word = 16-bits

Ex Range of integers

Determine the range of integers in base-10 that can be represented on a 16-bit computer.

Sol^n : First bit holds the sign. The other 15 bits from 0 to 111...111 (15 bits)

upper limit

$$(1\times 2^{14}) + (1\times 2^{13}) + \cdots + (1\times 2^{0}) = 32767 = (2^{15}-1)$$

Range is : -32767 to 32767

it is redundant to use $1000\ldots000$ (15 bits) "minus zero"

If is employed to represent an additional negative number :

-32768 to 32767

- Using 32 bits, largest integer $+2147483647$
- Signed magnitude method is not used to represent integers on conventional computers.
- 2's complement technique directly incorporates the sign into the number's magnitude rather than providing a separate bit to represent plus or minus.

Fixed-point versus Floating Point representation

- Most scientific computations are performed using real numbers, that is, numbers with a fractional part.
- Question 1: how many bits are needed to encode real number?
- Question 2: how to represent real numbers using these bits?
- In engineering normally at least 7 to 8 significant digits precision is required.
  - Hence number of bits to encode 8 decimal digits is approximately 26 (since $\log_2 10 = 3.32$ bits are needed on the average to encode a digit)
- In computers, numbers are stored as a sequence of 8-bit bytes.
- Thus for 26 bits, 32 bits (4 bytes) which is bigger than 26 bits is a logical size to use for real numbers.
- How do we break the 32 bits into an integer part and a fractional part?

One method is shown below:

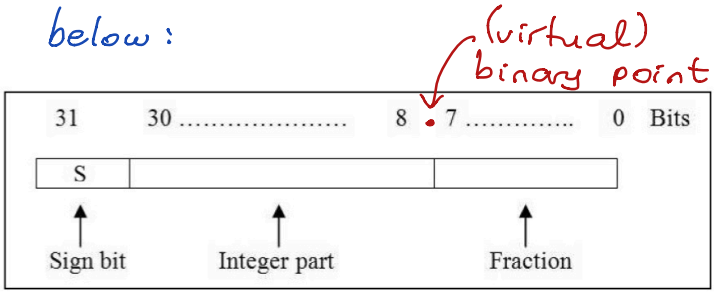


Figure 1. Fixed point representation of real numbers in binary using 32 bits.

with this fixed-pt representation, the largest and smallest positive binary numbers that may be represented using 32 bits are:

Largest: + 111 ..... 1.11111111 = 1677215.998046875
(23 bits, 8 bits)

Smallest: + 000 ... 0.00000001 = 0.00390625
(23 bits, 8 bits)

} range is small

- This range of real numbers, when fixed-pt representation is used, is not sufficient in many practical problems.
- Another representation called normalized floating point representation is used for real numbers in computers.
  - → 32 bits are divided into two parts:
    - → Mantissa with its sign (represents fractions)
    - → Exponent with its sign (power of 2 which multiplies the mantissa)

$$(\text{sign}) \times \text{mantissa} \times 2^{\pm \text{exponent}}$$

(sign): 1 bit; mantissa: binary fraction with a non-zero leading bit; exponent: binary integer

Q1. How many bits are to be used for the mantissa? (reserve 1 bit for the sign of the number)

Q2. How many bits are to be used for the exponent?

Q3. How to represent the sign of the exponent?

Answer Q1: to achieve at least 7 significant decimal digits, approx. 23 bits are needed for the mantissa

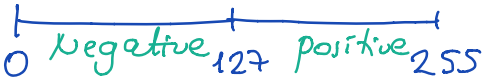
Answer Q2: ⇒ 8 bits for the exponent.

Answer Q3: Exponent may be represented in sign magnitude form. ⇒ Range of exponent: −127 to 127

Disadvantage: two representations for 0 exponent: +0 and −0

To avoid this one may use biased format to represent the exponent, where the exponent has no sign.

The range 0 to 255 of an 8-bit number is divided into two parts: 0 – 127 and 128 – 255

Exponent bits: 

$\Rightarrow$ Range of exponent -127 to $+128$.

The largest floating pt. numbers using 32 bits:

$$0.\underbrace{111\ldots111}_{23\text{ bits}} \times 2^{1111111}$$

$$\cong 3.4 \times 10^{38}$$

Smallest floating pt. number is:

$$0.\underbrace{100\ldots00}_{23\text{ bits}} \times 2^{-127} \cong 0.293 \times 10^{-38}$$

IEEE 754 Standard uses the most significant bit of mantissa as always 1, on the left of the (virtual) decimal point.

$$(-1)^S \times (1.f)_2 \times 2^{\text{exponent}-127}$$

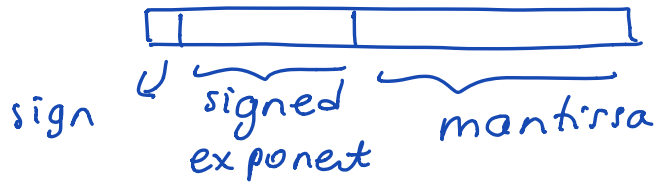
Floating-point Representation

- Fractional quantities will be represented in floating-pt. form.
  - fractional part (mantissa or significand)
  - integer part (exponent or characteristic)

e → exponent

m b → base of the number system used
↳ mantissa

Ex 156.78 $\longrightarrow 0.15678 \times 10^{3}$ in floating point base-10 system.



- mantissa is <u>normalized</u> if it has leading zero digits.

Ex $\frac{1}{34} = 0.029411765...$

- Floating pt base-10 system with 4 decimal places: 0.0294×10^{0}
- normalize to drop the leading zero:

0.2941×10^{-1}

↳ retain an additional significant figure.

- Due to normalization:

$$\frac{1}{b} \leq m < 1$$

b: base $\Rightarrow$ base 10: $0.1 \leq m < 1$

base 2: $0.5 \leq m < 1$

- <u>Advantage</u>: both fractions and very large numbers can be represented.

- Disadvantages: . take-up more room } than integers
  . takes longer to process }
  . mantissa has a finite # of significant figures ⇒ round-off error.

Ex Hypothetical set of floating-pt numbers store information using 7-bit words.

1st bit : sign of the number
2nd bit : " " exponent
3rd-4th bits : magnitude of the exponent
last three : " " mantissa

Smallest positive number:

2^1	2^0	2^{-1}	2^{-2}	2^{-3}		
0	1	1	1	1	0	0

: $(0.100)_2 \times 2^{-(11)_2}$

sign of the number
magnitude of exponent
magnitude of mantissa
sign of exponent

exponent : $(1 \times 2^1) + (1 \times 2^0) = 3 \Rightarrow$ exponent $= -3$

mantissa : $(1 \times 2^{-1}) + (0 \times 2^{-2}) + (0 \times 2^{-3}) = 0.5$

Smallest positive number is : $0.5 \times 2^{-3} = (0.0625)_{10}$

Next highest number:

$0111101 = (1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}) \times 2^{-3} = (0.078125)_{10}$

$0111110 = (1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3}) \times 2^{-3} = (0.093750)_{10}$

$01\,11\,111 = (1\times2^{-1} + 1\times2^{-2} + 1\times2^{-3})\times2^{-3} = (0.109375)_{10}$

spaced evenly with an interval of 0.015625

To continue increasing:

set exponent = 10 $\Rightarrow (1\times2^{1}) + (0\times2^{0}) = 2$

$\Rightarrow$ exponent $= -2$

set mantissa = 100

$01\,10\,100 = (0.125000)_{10}$

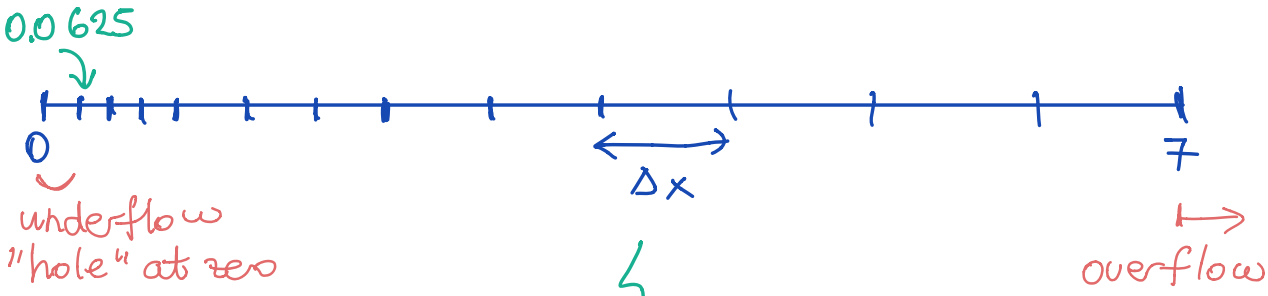
$01\,10\,101 = (0.156250)_{10}$ } 0.03125

$01\,10\,110 = (0.187500)_{10}$

$01\,10\,111 = (0.218750)_{10}$

Max. number

$0011111 = (7)_{10}$



the gap between numbers increase as the numbers grow in magnitude

Machine Epsilon: (Machine Precision)

There is a number $\varepsilon_m > 0$ such that

$1 + \delta = 1$, whenever $|\delta| < \varepsilon_m$ ↳ machine epsilon

Ex base $b = 2$

of mantissa bits $t = 3$

$$\boxed{\varepsilon_m = b^{1-t}}$$

$$\varepsilon = 2^{1-3} = 2^{-2} = 0.25$$

IEEE format: 24 bits → mantissa, 7 significant digits

range ~ $10^{-38} - 10^{39}$

Double precision: 15-16 decimal digits of precision

range: $10^{-308} - 10^{308}$

Arithmetic Manipulations of Computer Numbers

- Arithmetic operations can also cause round-off errors.
- Assume a hypothetical decimal computer with a 4-digit mantissa and a 1-digit exponent.
- we want to add: 0.1557×10^{1} and 0.4381×10^{-1}

  $0.4381 \times 10^{-1} \longrightarrow 0.004381 \times 10^{1}$ (shift the second number)

Add:

$$\begin{array}{r} 0.1557 \times 10^{1} \\ +\ 0.004381 \times 10^{1} \\ \hline 0.160081 \times 10^{1} \end{array}$$

$\longrightarrow$ chop the result to 0.1600×10^{1}

(lost digits: 81)

- To subtract 26.86 from 36.41

$$\begin{array}{r} 0.3641 \times 10^{2} \\ -\ 0.2686 \times 10^{2} \\ \hline 0.0955 \times 10^{2} \end{array} \xrightarrow{\text{normalize}} 0.9550 \times 10^{1}$$

(0 not significant, added to fill the empty space)

- Subtraction when two numbers are very close

$$\begin{array}{r} 0.7642 \times 10^{3} \\ -\ 0.7641 \times 10^{3} \\ \hline 0.0001 \times 10^{3} \end{array} \Longrightarrow 0.1 \times 10^{0} = 0.1000$$

- three non-significant zeros are appended
- Subsequent manipulations will act as if these zeros are significant.
- Greatest source of error in numerical methods.

- Multiplication

$(0.1363 \times 10^{3}) \times (0.6423 \times 10^{-1}) = 0.08754549 \times 10^{2}$

(add the exponents and multiply the mantissas)

↓ normalize

$\Rightarrow 0.8754549 \times 10^{1}$

chop $\Rightarrow 0.8754 \times 10^{1}$

- Adding a large and a small number

Add: 0.0010 and 4000

$$\begin{array}{r} 0.4000 \times 10^{4} \\ + \; 0.0000001 \times 10^{4} \\ \hline 0.4000001 \times 10^{4} \end{array}$$

↓ chop

0.4000×10^{4}

(a hypothetical computer with 4 digit mantissa and 1 digit exponent)

↓

no effect of addition.

- This type error can occur in the computation of infinite series. The initial terms are large as compared to later terms.

  Solution: add the small terms first, and then add the large numbers.